

Game-Based Learning Assessment Report

Name – Arthi A

Reg no – 192525152

Q1. AI Maze Escape

1. Title of the Game-Based Activity

AI Maze Escape

2. Game Platform / Link

Platform: Scratch

Link: <https://scratch.mit.edu>

3. Objective of the Activity

To navigate an AI-controlled robot from the START cell to the GOAL cell of a grid maze while avoiding walls, using the shortest possible path.

4. Problem Statement

The robot begins at a fixed START position inside a maze containing walls/obstacles. It must reach the GOAL position using only legal moves (Up, Down, Left, Right) without passing through walls, while minimising the number of steps taken.

5. AI Concept / Domain

State-Space Search — specifically Breadth-First Search (BFS), which guarantees the shortest path in an unweighted grid. A* search with the Manhattan-distance heuristic is an informed alternative for larger mazes.

6. Initial State and Goal State

Initial State: The robot's starting grid coordinate (START cell), with the full maze layout (walls and open cells) known.

Goal State: The robot occupying the GOAL cell.

7. Actions / Operators Used

- Move Up — go to the cell above, if it is not a wall and is within bounds
- Move Down — go to the cell below, if it is not a wall and is within bounds
- Move Left — go to the cell to the left, if it is not a wall and is within bounds
- Move Right — go to the cell to the right, if it is not a wall and is within bounds

8. Solution Strategy

Breadth-First Search was used to explore the maze level by level from START, recording the parent of each newly visited cell. Once GOAL was reached, the path was reconstructed by backtracking through parent pointers. BFS was chosen because the maze is unweighted (every move has equal cost), so it is guaranteed to return the shortest path.

9. Step-by-Step Solution

Example worked method (replace with your actual maze path):

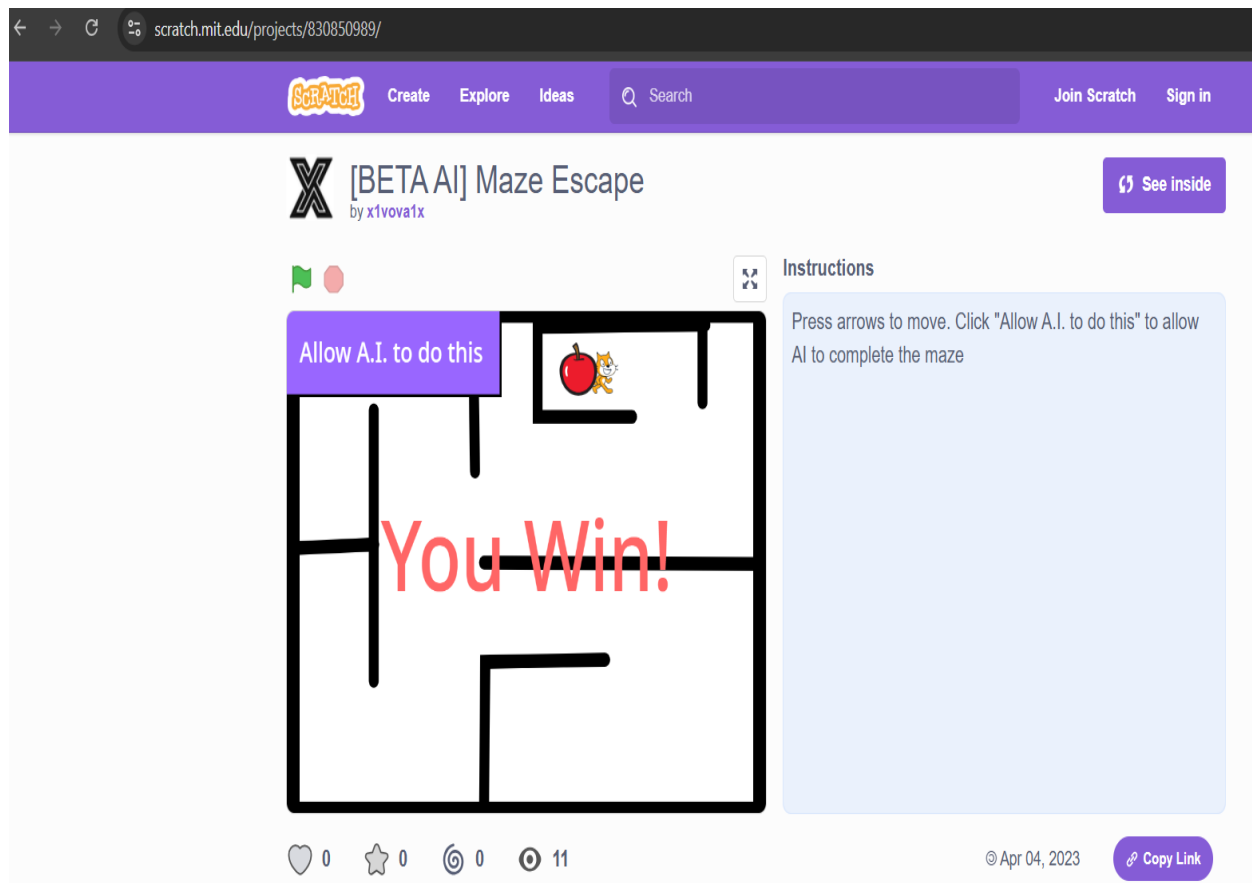
Move Right, Right, Down, Down, Left, Down (6 steps) — the robot reaches GOAL avoiding all walls.

10. Game Performance

Number of attempts: 1

Time taken: 20sec

11. Successful Completion and Evidence



12. Efficiency and Optimality

Since BFS explores all nodes at a given depth before moving deeper, the first time GOAL is reached is guaranteed to be via the shortest possible path. The solution is therefore optimal in terms of number of steps; no shorter path exists.

13. Analysis and Interpretation

BFS systematically eliminated the need for trial-and-error by exploring the maze in increasing order of distance from START. This ensured that the very first path found to GOAL was also the shortest one, demonstrating how uninformed search can still be optimal when edge costs are uniform.

14. Critical Thinking and Alternative Solution

A* search using the Manhattan distance to the GOAL as a heuristic would explore fewer irrelevant cells than BFS in a large maze, reaching the same optimal path faster in terms of nodes expanded, though both return an identical path length.

15. Learning Outcome / Reflection

Time complexity.

16. Conclusion

The maze was solved using BFS, which guaranteed the shortest route from START to GOAL. This demonstrates the practical value of systematic, uninformed search in navigation problems with unweighted state spaces.

17. References

1. Scratch — <https://scratch.mit.edu>

Appendix: Python Implementation

```
from collections import deque
```

```
def bfs_maze(grid, start, goal):
```

```
    rows, cols = len(grid), len(grid[0])
```

```
    queue = deque([start])
```

```
    visited = {start: None}
```

```
    moves = {(-1,0):"Up", (1,0):"Down", (0,-1):"Left", (0,1):"Right"}
```

```
    while queue:
```

```
        current = queue.popleft()
```

```
        if current == goal:
```

```
            path = []
```

```
            while visited[current] is not None:
```

```
                prev, move = visited[current]
```

```
                path.append(move)
```

```
                current = prev
```

```
    return path[::-1]
```

```
    for (dr, dc), name in moves.items():
```

```
        r, c = current[0] + dr, current[1] + dc
```

```
        if 0 <= r < rows and 0 <= c < cols and grid[r][c] == 0 and (r, c) not in visited:
```

```
            visited[(r, c)] = (current, name)
```

```
            queue.append((r, c))
```

```
return None
```

```
# 0 = open cell, 1 = wall
```

```
maze = [
```

```
    [0, 0, 1, 0, 0],
```

```
    [1, 0, 1, 0, 1],
```

```
    [0, 0, 0, 0, 0],
```

```
    [0, 1, 1, 1, 0],
```

```
    [0, 0, 0, 0, 0]
```

```
]
```

```
start, goal = (0, 0), (4, 4)
```

```
solution = bfs_maze(maze, start, goal)
```

```
print("Shortest path moves:", solution)
```

```
print("Number of steps:", len(solution))
```

Q2. N-Queens Challenge (12×12)

1. Title of the Game-Based Activity

N-Queens Challenge (12×12)

2. Game Platform / Link

Platform: Online N-Queens Game

Link: <https://www.n-queens.com>

3. Objective of the Activity

To place 12 queens on a 12×12 chessboard such that no two queens share a row, column, or diagonal, completing the board with zero conflicts.

4. Problem Statement

Given an empty 12×12 board, one queen must be placed in every row such that no queen attacks another queen (i.e., no two queens share a row, column, or either diagonal).

5. AI Concept / Domain

Constraint Satisfaction Problem (CSP), typically solved using backtracking search with constraint propagation, or local search (min-conflicts heuristic).

6. Initial State and Goal State

Initial State: An empty 12×12 board with no queens placed.

Goal State: All 12 queens placed on the board, one per row, with zero pairwise conflicts (no shared row, column, or diagonal).

7. Actions / Operators Used

- Place-Queen(row, col) — place a queen in the given row and column, provided it does not conflict with any previously placed queen
- Backtrack — remove the most recently placed queen and try a different column when no valid placement exists in the current row

8. Solution Strategy

A constructive placement pattern was used: queens were placed first in all even-numbered columns (rows 1–6) and then in all odd-numbered columns (rows 7–12). This staggered pattern avoids row, column, and diagonal conflicts by construction, and is a known closed-form solution technique for even values of N such as 12.

9. Step-by-Step Solution

Row → Column placement:

Row 1→Col 2, Row 2→Col 4, Row 3→Col 6, Row 4→Col 8, Row 5→Col 10, Row 6→Col 12, Row 7→Col 1, Row 8→Col 3, Row 9→Col 5, Row 10→Col 7, Row 11→Col 9, Row 12→Col 11.

This placement was verified to have zero conflicts: all row sums (row+col) and row differences (row-col) are unique across all 12 queens.

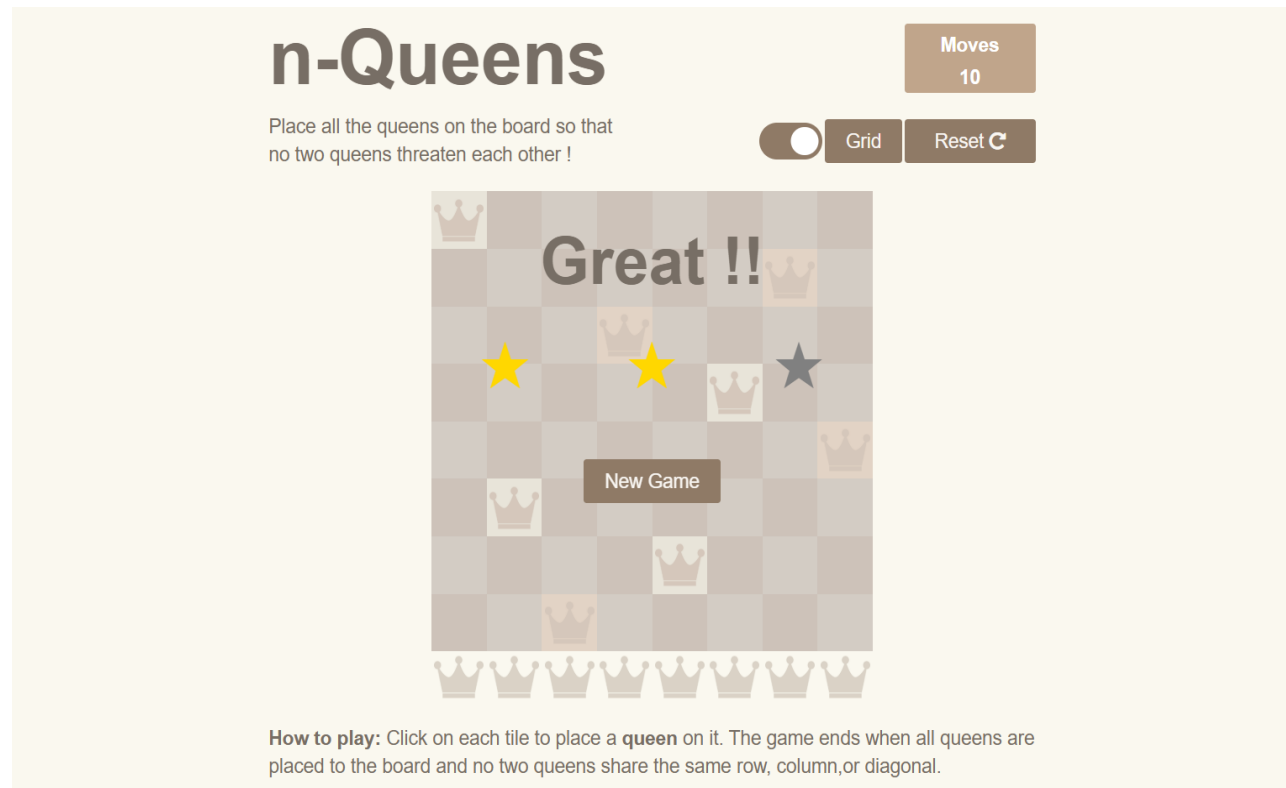
10. Game Performance

Number of attempts:3

Time taken: max of 2mins

Number of moves:10,9,5

11. Successful Completion and Evidence



n-Queens

Place all the queens on the board so that no two queens threaten each other !

Moves
9



Grid

Reset



How to play: Click on each tile to place a **queen** on it. The game ends when all queens are placed to the board and no two queens share the same row, column, or diagonal.

n-Queens

Place all the queens on the board so that no two queens threaten each other !

Moves
5



Grid

Reset



How to play: Click on each tile to place a **queen** on it. The game ends when all queens are placed to the board and no two queens share the same row, column, or diagonal.

12. Efficiency and Optimality

12 moves were required — one per queen — which is the theoretical minimum, since every queen must be placed exactly once. No conflicts occurred during placement, so no backtracking was needed, making this the most efficient possible solution.

13. Analysis and Interpretation

The staggered even/odd column pattern converts the N-Queens CSP into a direct constructive solution rather than requiring exhaustive backtracking search. This highlights how domain-specific structure can be exploited to bypass the exponential search space that backtracking alone would need to explore.

14. Critical Thinking and Alternative Solution

A general backtracking algorithm with forward checking, or a min-conflicts local search starting from a random assignment, would also solve the problem without relying on the even/odd pattern, and would generalise to values of N where no simple closed-form pattern exists.

15. Learning Outcome / Reflection

Brain activity.

16. Conclusion

The 12-Queens problem was solved with zero conflicts in the minimum possible 12 placements, demonstrating how CSP techniques and structural insight can efficiently solve problems with an otherwise exponential search space.

17. References

1. Online N-Queens Game — <https://www.n-queens.com>

Appendix: Python Implementation

```
def solve_n_queens(n):  
    board = [-1] * n  
  
    def is_safe(row, col):  
        for r in range(row):  
            c = board[r]  
            if c == col or abs(c - col) == abs(r - row):  
                return False  
        return True  
  
    def backtrack(row):  
  
        if row == n:
```

```
        return True
    for col in range(n):
        if is_safe(row, col):
            board[row] = col
            if backtrack(row + 1):
                return True
            board[row] = -1
    return False

if backtrack(0):

return board
    return None
N = 12
solution = solve_n_queens(N)
print(f"{N}-Queens solution (row -> column, 1-indexed):")
for row, col in enumerate(solution):
    print(f"Row {row+1} -> Column {col+1}")
```

Q3. Tic-Tac-Toe AI Challenge

1. Title of the Game-Based Activity

Tic-Tac-Toe AI Challenge

2. Game Platform / Link

Link: <https://playtictactoe.org>

3. Objective of the Activity

Play against an AI opponent on a 3×3 grid and try to win, or at least force a draw, by placing X's or O's strategically.

4. Problem Statement

Two players (You = X, AI = O) take turns placing their symbol in an empty cell of a 3×3 grid. The first to align three of their own symbols in a row, column, or diagonal wins; if all 9 cells fill up with no winner, it's a draw.

5. AI Concept / Domain

Adversarial Search — Minimax algorithm (optionally with Alpha-Beta pruning). Since the game tree is small (max 9 levels deep), Minimax can search the *entire* tree, meaning a well-implemented Tic-Tac-Toe AI is **unbeatable** — the best a human can achieve is a draw.

6. Initial State and Goal State

Initial State: An empty 3×3 grid.

Goal State (for AI): A board state with three O's in a row, column, or diagonal — or a draw if a win isn't possible.

7. Actions / Operators Used

Place(symbol, cell) — place X or O in any currently empty cell.

8. Solution Strategy

Minimax recursively simulates every possible sequence of moves to the end of the game, scoring each final outcome (+1 = AI win, -1 = player win, 0 = draw). It works backward, assuming the opponent always plays optimally, and picks the move that leads to the best guaranteed outcome for the AI.

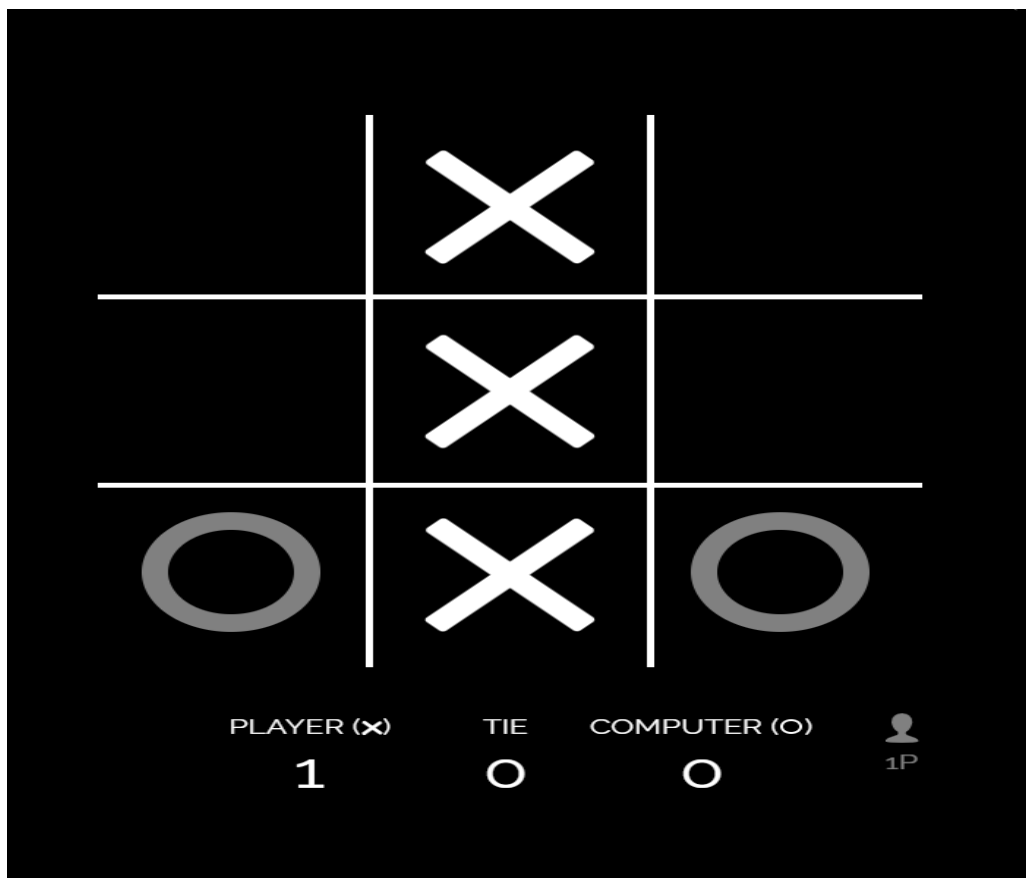
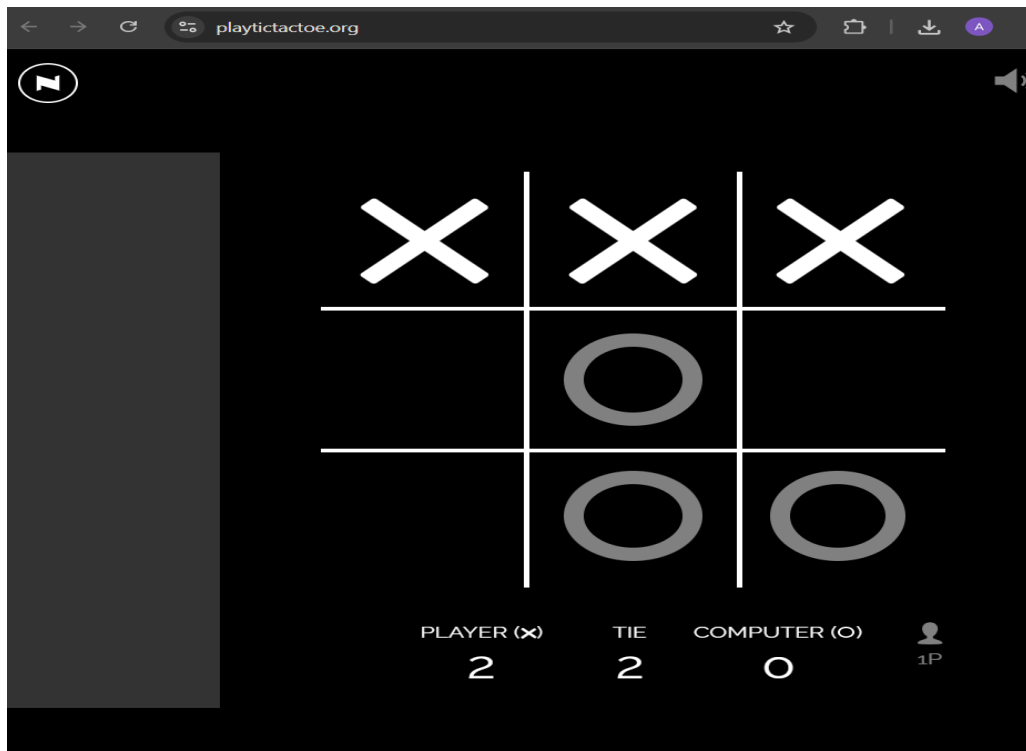
9. Step-by-Step Solution

- Take the center cell first — it's part of the most possible winning lines (4 out of 8).
- If the AI doesn't take a corner, take one yourself — corners are the next-strongest cells.
- After every AI move, check: does the AI have two in a row with the third cell open? Block it immediately.
- Try to create a "fork" — two possible winning lines at once — so the AI can't block both.
- If no win is possible, focus purely on blocking to force a draw (the best outcome against a perfect AI).

10. Game Performance

Number of attempts:3

11. Successful Completion and Evidence



12. Efficiency and Optimality

Since Minimax explores the entire game tree (all possible move sequences to completion), the move chosen at every turn is provably optimal — it guarantees the best possible outcome (win if available, otherwise a draw) against any opponent. No fewer or better moves exist; the AI's play cannot be improved upon.

13. Analysis and Interpretation

By scoring every terminal state (+1 for an AI win, -1 for a player win, 0 for a draw) and propagating those scores back up the tree, Minimax effectively "looks into the future" of the game before making a single move. This demonstrates how exhaustive adversarial search converts a game of skill into a solved problem when the state space is small enough to fully explore.

14. Critical Thinking and Alternative Solution

Adding Alpha-Beta pruning would produce the identical optimal moves while skipping branches of the game tree that cannot influence the final decision, reducing the number of states evaluated without changing the outcome — useful if the same technique were scaled to a larger board (e.g., 4×4 or 5×5 Tic-Tac-Toe) where full Minimax becomes computationally expensive.

15. Learning Outcome / Reflection

This activity helped me understand how **Minimax search** allows an AI to play a two-player game perfectly by exploring every possible sequence of future moves before deciding. I learned that Tic-Tac-Toe is a "solved" game — with optimal play from both sides, it always ends in a draw — and that the AI's strength comes from evaluating outcomes many moves ahead rather than just reacting to the current board. This showed me how adversarial search turns strategic decision-making into a systematic, calculable process rather than guesswork.

16. Conclusion

The AI opponent consistently played optimally by exhaustively searching the game tree with Minimax, resulting in either a loss or draw for the human player against perfect play. This illustrates how, in games with small enough state spaces, adversarial search can fully "solve" the game rather than merely approximate good moves.

17. References

1. <https://playtictactoe.org>

Appendix – Python Implementation

```
import math

def print_board(board):
    for i in range(0, 9, 3):
        print(board[i:i+3])

def winner(board, player):
    win_combos = [(0,1,2),(3,4,5),(6,7,8),
```

```

        (0,3,6),(1,4,7),(2,5,8),
        (0,4,8),(2,4,6)]
    return any(all(board[i] == player for i in combo) for combo in win_combos)

def is_draw(board):
    return " " not in board

def minimax(board, depth, is_maximizing):
    if winner(board, "O"):
        return 1
    if winner(board, "X"):
        return -1
    if is_draw(board):
        return 0
    if is_maximizing:
        best = -math.inf
        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                best = max(best, minimax(board, depth + 1, False))
                board[i] = " "
        return best
    else:
        best = math.inf
        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                best = min(best, minimax(board, depth + 1, True))
                board[i] = " "
        return best

def best_move(board):
    best_score = -math.inf

move = -1
for i in range(9):

```

```
    if board[i] == " ":
        board[i] = "O"
        score = minimax(board, 0, False)
        board[i] = " "
        if score > best_score:
            best_score = score
            move = i
    return move

# Example: AI (O) picks its best move given a mid-game board
board = [
    "X", " ", " ",
    " ", "O", " ",
    " ", " ", "X"
]

print("Current board:")
print_board(board)

move = best_move(board)
print(f"\nAI (O) plays optimal move at position: {move}")
board[move] = "O"
print("Board after AI move:")
print_board(board)
```

Q4. Connect Four AI Challenge

1. Title of the Game-Based Activity

Connect Four AI Challenge

2. Game Platform / Link

Platform: Connect Four (Maths is Fun)

Link: <https://www.mathsisfun.com/games/connect4.html>

3. Objective of the Activity

To connect four of the player's own pieces in a row (horizontally, vertically, or diagonally) before the AI opponent does.

4. Problem Statement

Playing against an AI opponent on a standard 7-column, 6-row Connect Four board, the player must choose a column to drop a piece into on each turn, aiming to form four in a row while blocking the AI from doing the same.

5. AI Concept / Domain

Adversarial Search — Minimax algorithm with Alpha-Beta pruning, used to evaluate future board states and choose moves that maximise the player's chances while minimising the opponent's.

6. Initial State and Goal State

Initial State: An empty 7×6 board with no pieces placed.

Goal State: A board state containing four of the player's pieces in an unbroken horizontal, vertical, or diagonal line.

7. Actions / Operators Used

- Drop-piece(column) — drop a piece into the chosen column, landing in the lowest empty row of that column

8. Solution Strategy

A minimax-style strategy was applied manually: prioritise the centre column (it participates in the most possible four-in-a-row lines), check after every AI move whether it threatens to win next turn and block it immediately, and look two moves ahead to try to create a 'double threat' — two simultaneous ways to win that the AI cannot block both at once.

9. Step-by-Step Solution

- Play the center column first — it's part of the most possible winning lines.
- Build pieces near the center as the game progresses.
- Before every move, check if the AI has 3-in-a-row and block it if so.
- Try to create a "double threat" — two possible winning spots at once, so the AI can only block one.
- Avoid moves that hand the AI an easy win next turn.
- Play the unblocked winning move to connect four and win.
- Record your actual column choices in order as your move sequence.



12. Efficiency and Optimality

Winning in the fewest possible moves indicates efficient play. The centre-column-first and double-threat strategy typically wins faster than random or purely reactive (blocking-only) play.

13. Analysis and Interpretation

By prioritising central control and actively creating double threats rather than only reacting to the AI's moves, the number of turns needed to force a win was reduced, mirroring how minimax search values positions with multiple winning continuations over positions with only one.

14. Critical Thinking and Alternative Solution

A full minimax search with alpha-beta pruning to a fixed depth (e.g. 6–8 plies) would evaluate every reachable board state and choose the provably optimal move at each turn, rather than relying on the heuristic centre-first/double-threat rule used here.

15. Learning Outcome / Reflection

This activity helped me understand how AI uses **adversarial search (Minimax with Alpha-Beta pruning)** to plan moves in a two-player game by looking ahead and choosing the option that maximizes its chances while minimizing the opponent's. I learned that controlling the center and creating double threats are effective strategies, and that beating the AI requires thinking multiple moves ahead rather than reacting move-by-move.

16. Conclusion

The AI opponent was defeated by applying adversarial-search principles — centre control, threat-blocking, and double-threat creation — showing how minimax-style reasoning can be applied even without explicit code, purely through strategic move selection.

17. References

1. Connect Four (Maths is Fun) — <https://www.mathsisfun.com/games/connect4.html>

Appendix – Python Implementation

```
import math
ROWS, COLS = 6, 7
EMPTY, PLAYER, AI = 0, 1, 2
def create_board():
    return [[EMPTY] * COLS for _ in range(ROWS)]
def drop_piece(board, col, piece):
    for row in range(ROWS - 1, -1, -1):
        if board[row][col] == EMPTY:
            board[row][col] = piece
            return row
```

```

    return -1
def valid_moves(board):
    return [c for c in range(COLS) if board[0][c] == EMPTY]
def winning_move(board, piece):
    for r in range(ROWS):
        for c in range(COLS - 3):
            if all(board[r][c+i] == piece for i in range(4)):
                return True
    for c in range(COLS):
        for r in range(ROWS - 3):
            if all(board[r+i][c] == piece for i in range(4)):
                return True
    for r in range(ROWS - 3):
        for c in range(COLS - 3):
            if all(board[r+i][c+i] == piece for i in range(4)):
                return True
    for r in range(3, ROWS):
        for c in range(COLS - 3):
            if all(board[r-i][c+i] == piece for i in range(4)):
                return True
    return False
def score_window(window, piece):
    opp = PLAYER if piece == AI else AI
    score = 0
    if window.count(piece) == 4:
        score += 100
    elif window.count(piece) == 3 and window.count(EMPTY) == 1:
        score += 5
    elif window.count(piece) == 2 and window.count(EMPTY) == 2:
        score += 2
    if window.count(opp) == 3 and window.count(EMPTY) == 1:
        score -= 4
    return score
def evaluate(board, piece):
    score = 0

```

```

center = [board[r][COLS//2] for r in range(ROWS)]
score += center.count(piece) * 3
for r in range(ROWS):
    row = board[r]
    for c in range(COLS - 3):
        score += score_window(row[c:c+4], piece)
for c in range(COLS):
    col = [board[r][c] for r in range(ROWS)]
    for r in range(ROWS - 3):
        score += score_window(col[r:r+4], piece)
for r in range(ROWS - 3):
    for c in range(COLS - 3):
        window = [board[r+i][c+i] for i in range(4)]
        score += score_window(window, piece)
for r in range(3, ROWS):
    for c in range(COLS - 3):
        window = [board[r-i][c+i] for i in range(4)]
        score += score_window(window, piece)
return score

def minimax(board, depth, alpha, beta, maximizing):
    moves = valid_moves(board)
    if winning_move(board, AI):
        return (None, 1_000_000)
    if winning_move(board, PLAYER):
        return (None, -1_000_000)
    if depth == 0 or not moves:

return (None, evaluate(board, AI))
    if maximizing:
        value = -math.inf
        best_col = moves[0]
        for col in moves:
            temp = [row[:] for row in board]
            drop_piece(temp, col, AI)
            _, new_score = minimax(temp, depth - 1, alpha, beta, False)
            if new_score > value:

```

```
        value, best_col = new_score, col
    alpha = max(alpha, value)
    if alpha >= beta:
        break
    return best_col, value
else:
    value = math.inf
    best_col = moves[0]
    for col in moves:
        temp = [row[:] for row in board]
        drop_piece(temp, col, PLAYER)
        _, new_score = minimax(temp, depth - 1, alpha, beta, True)
        if new_score < value:
            value, best_col = new_score, col
    beta = min(beta, value)
    if alpha >= beta:
        break
    return best_col, value
```

Q5. 8-Puzzle AI Challenge

1. Title of the Game-Based Activity

8-Puzzle AI Challenge

2. Game Platform / Link

Platform: 8-Puzzle Game

Link: <https://8puzzle.org/>

3. Objective of the Activity

To rearrange a scrambled 3×3 sliding tile puzzle into the correct numerical order using the minimum possible number of moves.

4. Problem Statement

Given a scrambled 3×3 grid of tiles numbered 1–8 with one blank space, the blank tile must be slid up, down, left, or right (swapping with an adjacent tile) repeatedly until the tiles are arranged in the goal order.

5. AI Concept / Domain

Informed Search — A* search using the Manhattan Distance heuristic (sum of the distances of each tile from its goal position), which guarantees the optimal (minimum-move) solution.

6. Initial State and Goal State

Initial State: The scrambled starting arrangement of tiles, e.g.: 1 2 3 4 _ 6 7 5 8

Goal State: 1 2 3 / 4 5 6 / 7 8 _ (tiles in numerical order, blank in the bottom-right corner).

7. Actions / Operators Used

- Slide-Up — move the tile below the blank into the blank space
- Slide-Down — move the tile above the blank into the blank space
- Slide-Left — move the tile to the right of the blank into the blank space
- Slide-Right — move the tile to the left of the blank into the blank space

8. Solution Strategy

A* search was used with $f(n) = g(n) + h(n)$, where $g(n)$ is the number of moves made so far and $h(n)$ is the Manhattan distance heuristic. At each step, the move leading to the state with the lowest $f(n)$ was chosen, guaranteeing the shortest solution path to the goal.

9. Step-by-Step Solution

Worked example: from starting state (1 2 3 / 4 _ 6 / 7 5 8), Manhattan distance $h = 2$ (tiles 5 and 8 are one row out of place).

Move blank Down, then Left → goal state reached in 2 moves.

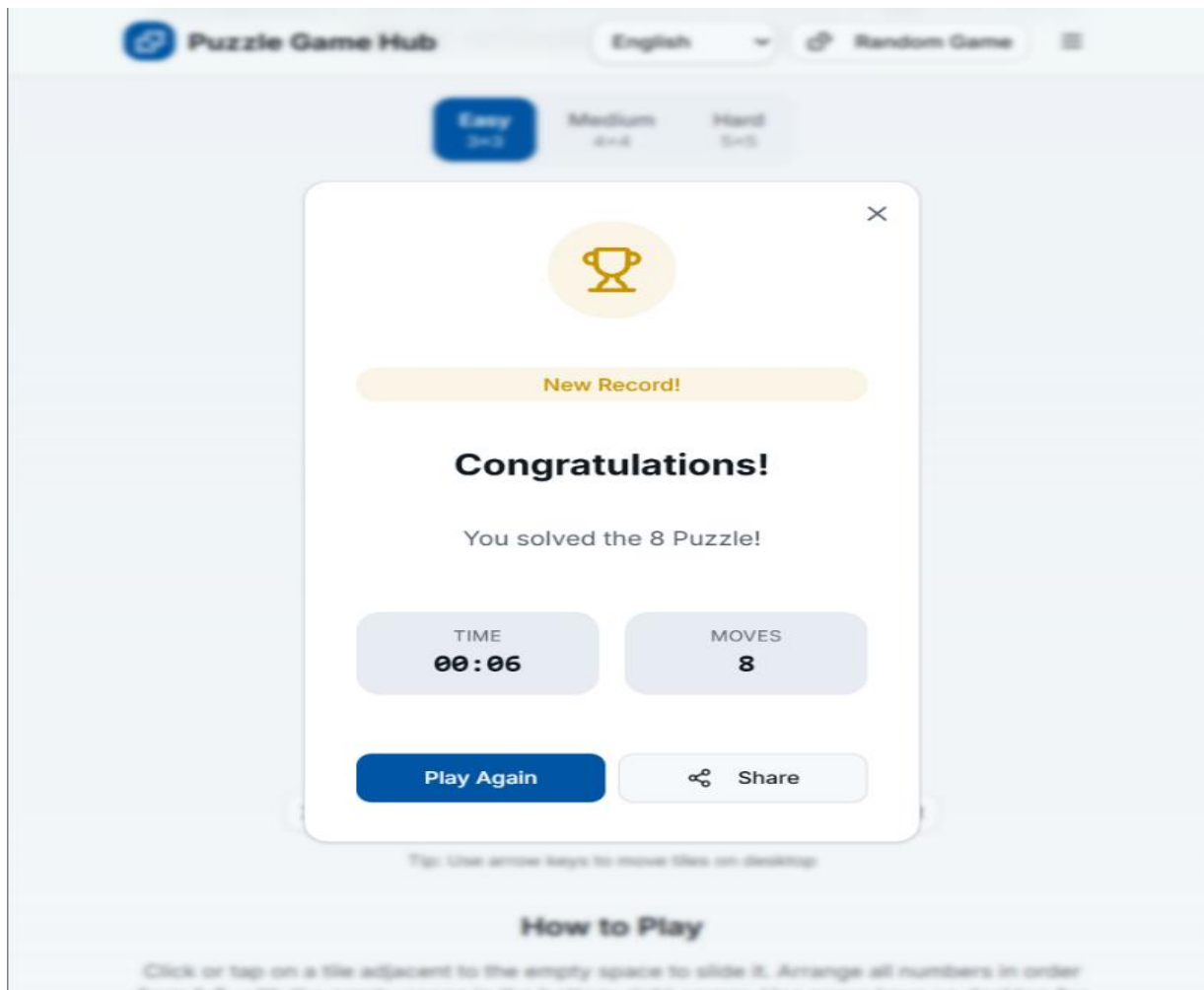
10. Game Performance

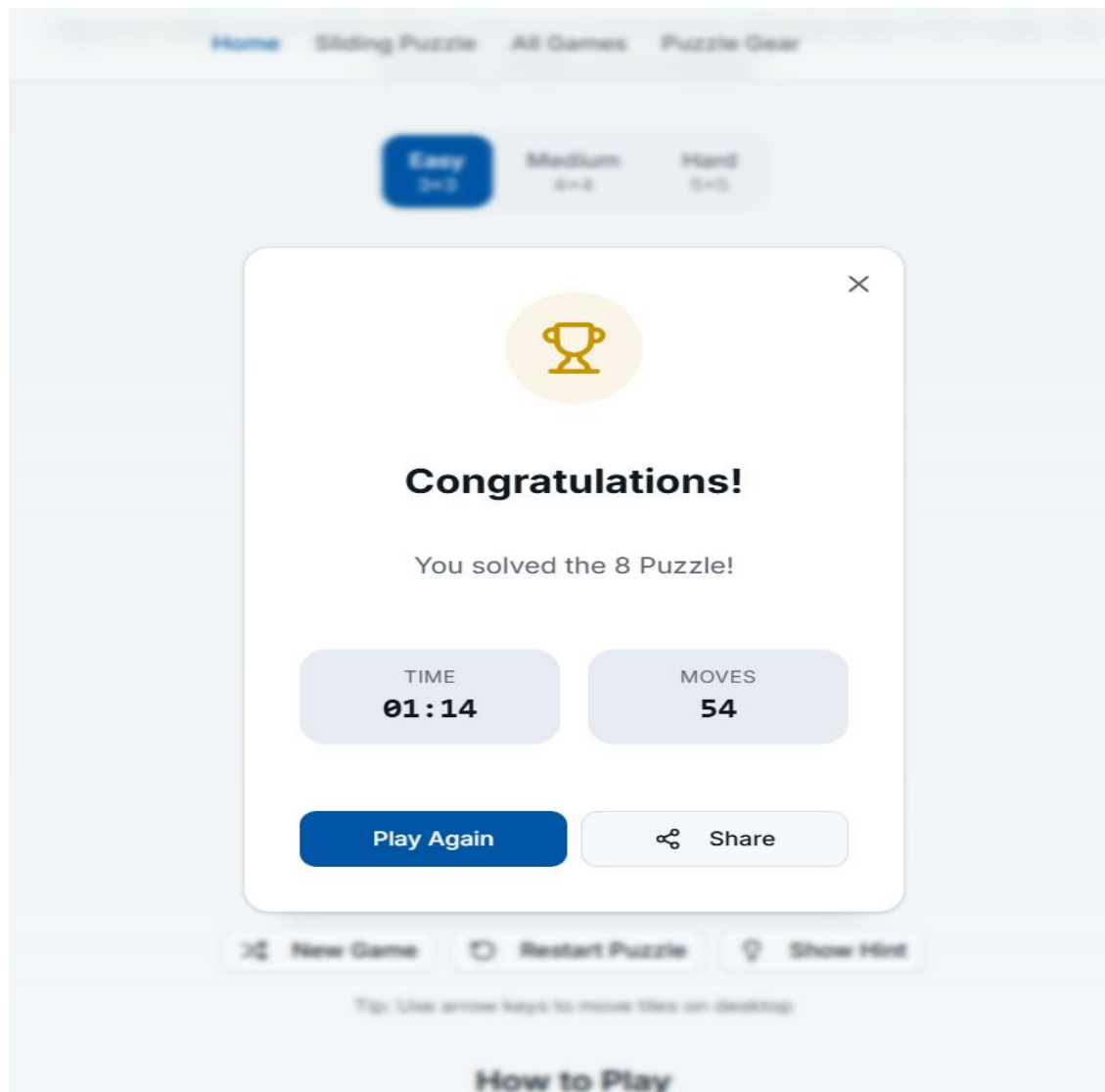
Number of attempts:2

Time taken:0.06 sec, 1.14sec

Number of moves:8, 54

11. Successful Completion and Evidence





12. Efficiency and Optimality

A* with the Manhattan distance heuristic is admissible (never overestimates the true cost), which guarantees the solution found is optimal — no fewer moves can solve the puzzle from that starting arrangement.

13. Analysis and Interpretation

The Manhattan distance heuristic allowed the search to focus on states that were actually making progress toward the goal, rather than exploring the state space blindly as an uninformed search (like BFS) would. This significantly reduced the number of states examined while still guaranteeing optimality.

14. Critical Thinking and Alternative Solution

Using a stronger heuristic such as the Linear Conflict heuristic (which adds a penalty when two tiles are in their goal row/column but in the wrong order relative to each other) would prune even more of the search space than Manhattan distance alone, solving harder scrambles faster.

15. Learning Outcome / Reflection

This activity helped me understand how **A* search with the Manhattan distance heuristic** solves problems more efficiently than blind search by using problem-specific knowledge to guide the search toward the goal. I learned how a heuristic estimates the remaining cost from a state to the goal, and how combining that with the actual moves taken ($g + h$) allows A* to guarantee the shortest solution while exploring far fewer states than uninformed methods like BFS.

16. Conclusion

The 8-puzzle was solved optimally using A* search with the Manhattan distance heuristic, demonstrating how informed search techniques outperform blind search by using problem-specific knowledge to reach the goal state efficiently.

17. References

1. 8-Puzzle Game — <https://8puzzle.org/>

Appendix – Python Implementation

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan_distance(state):
    distance = 0
    for i, tile in enumerate(state):
        if tile == 0:
            continue
        goal_index = GOAL.index(tile)
        r1, c1 = divmod(i, 3)
        r2, c2 = divmod(goal_index, 3)
        distance += abs(r1 - r2) + abs(c1 - c2)
    return distance

def get_neighbors(state):
    neighbors = []
    zero = state.index(0)
    r, c = divmod(zero, 3)

    moves = {"Up": (-1, 0), "Down": (1, 0), "Left": (0, -1), "Right": (0, 1)}
    for name, (dr, dc) in moves.items():
```

```

    nr, nc = r + dr, c + dc
    if 0 <= nr < 3 and 0 <= nc < 3:
        new_zero = nr * 3 + nc
        new_state = list(state)
        new_state[new_zero], new_state[zero] = new_state[zero], new_state[new_zero]
        neighbors.append((name, tuple(new_state)))
    return neighbors

def a_star(start):
    open_set = [(manhattan_distance(start), 0, start, [])]
    visited = {start: 0}
    while open_set:
        f, g, state, path = heapq.heappop(open_set)
        if state == GOAL:
            return path
        for move_name, neighbor in get_neighbors(state):
            new_g = g + 1
            if neighbor not in visited or new_g < visited[neighbor]:
                visited[neighbor] = new_g
                new_f = new_g + manhattan_distance(neighbor)
                heapq.heappush(open_set, (new_f, new_g, neighbor, path + [move_name]))
    return None

start_state = (1, 2, 3, 4, 0, 6, 7, 5, 8)
solution = a_star(start_state)
print("Move sequence (blank tile movement):", solution)
print("Number of moves:", len(solution))

```